Разработка модулей ядра Linux: Часть 4. Ядро и модуль

Олег Цилюрик
преподаватель тренингового отделения
Global Logic

27.04.2012

В этой статье приводится классификация архитектур ядра и описываются стандартные ошибки, возникающие при загрузке модуля.

Введение

В данной статье даются начальные сведения о взаимодействиях между модулем и ядром. Но прежде следует закончить обсуждение вопросов тестирования модулей ядра, рассмотрев стандартные ошибки, возникающие при загрузке модуля.

Ошибки, возникающие при загрузке модуля

При нормальной загрузке модуля командой insmod не выводится никаких сообщений. Но если в ходе загрузки возникает ошибка, то выводится сообщение, а модуль не будет загружен в состав ядра. Ниже перечислены ошибки, чаще всего встречающиеся при неудачной загрузке модуля, и их толкование:

- **insmod: can't read './params': No such file or directory** — неверно указан путь к файлу модуля, возможно, в указании имени файла не включено стандартное расширение файла модуля (***.ko**), а это обязательное требование.
- **insmod: error inserting './params.ko': -1 Operation not permitted** — наиболее вероятная причина: не хватает прав root, чтобы выполнить действия по установке модулей. Другая причина этого же сообщения: функция инициализации модуля возвратила ненулевое значение, но нередко такое завершение планируется преднамеренно, особенно на этапах отладки модуля.
- **insmod: error inserting './params.ko': -1 Invalid module format** — модуль скомпилирован для другой версии ядра, в данном случае модуль следует перекомпилировать. Это ошибка почти наверняка возникнет, если перенести любой работающий пример модуля на другой компьютер и попытаться загрузить его: совпадение реализаций — почти невероятно.
- **insmod: error inserting './params.ko': -1 File exists** — модуль с таким именем уже загружен, попытка загрузить модуль повторно.

© Copyright IBM Corporation 2012

- **insmod: error inserting './params.ko': -1 Invalid parameters** — модуль запускается с указанным параметром, тип которого не соответствует ожидаемому.

Ошибка (сообщение) может возникнуть и при попытке выгрузить модуль. Более того, так как прототип функции выгрузки модуля не предполагает возможности вернуть код неудачного завершения, то все сообщения могут поступать только от подсистемы управления модулями операционной системы. Наиболее часто при попытке выгрузить модуль возникают следующие ошибки:

ERROR: Removing 'params': Device or resource busy — счётчик ссылок модуля не равен нулю, в системе (возможно) есть модули, зависимые от данного. Но не исключено и то, что счетчик ссылок был инкрементирован в самом коде, а затем не декрементирован назад. Такая же ситуация может возникнуть после аварийного сообщения ядра Ooops... после загрузки модуля (ошибка в коде модуля в ходе отработки).

ERROR: Removing 'params': Operation not permitted — самая частая причина такого сообщения — не хватает полномочий root, чтобы выполнить операцию rmmod. Более экзотический случай появления такого сообщения: когда в коде модуля вообще забыли прописать функцию выгрузки **module_exit()**. В этом случае, в списке модулей можно видеть довольно редкий квалификатор **permanent** (это значит, что был создан **невыгружаемый** модуль и поможет только перезагрузка системы):

```
$ /sbin/lsmod | head -n2
Module                  Size  Used by
params                  6412  0 [permanent]
...
```

Ядро: монолитное и микроядро

Исторически сложилось, что все операционные системы, начиная с самых первых и до самых современных, принадлежат к одному из двух классов, которые различаются **по архитектуре ядра**:

- монолитное ядро (этот тип архитектуры появился первым, другие названия: моноядро, макроядро) — к этому классу относится большая часть известных операционных систем, например: IBM OS/360, RSX-11, VAX-VMS, MS-DOS, Windows (все модификации), OS/2, Linux, все клоны BSD (FreeBSD, NetBSD, OpenBSD), Sun Solaris;
- микроядро (эта архитектура возникла позже, другие названия: клиент-серверные операционные системы и системы с обменом сообщениями) — к этому классу относятся, например: QNX, MINIX 3, HURD, ядро Darwin MacOS, семейство ядер L4.

Понимание принципиальных различий между данными классами архитектур может помочь в осознании природы тех проблем, которые приходится решать при проектировании модулей ядра Linux.

В микроядерной архитектуре прикладное приложение A, которому необходим доступ к некоторому сервису, отправляет сообщение со своим запросом микроядру (например,

выполняя вызов **SendMessage()** в терминологии ОС QNX). Роль микроядра заключается только в том, чтобы ретранслировать запрос (в соответствии с его адресом) процессу Б (серверу, обслуживающему сервис, или драйверу). Процесс Б является таким же процессом **пространства пользователя**, как и прикладной процесс А. Выполнив запрос, процесс Б возвращает ответное сообщение (выполняя вызов **ReplyMessage()**), содержащее **результат** операции. Это ответное сообщение процесс Б передает микроядру, которое, в свою очередь, ретранслирует его процессу А, ожидающему ответа. Поэтому такая последовательность и называется: **синхронный** обмен сообщениями. Никакого двойного копирования данных при этом не происходит, так как супервизорное ядро обладает необходимыми полномочиями, чтобы копировать данные из адресного пространства процесса А и в пространство процесса Б и обратно. В такой системе ядро обслуживает минимально возможный набор системных вызовов (обычно менее 100), но главным его преимуществом является то, что драйверы (сервера) работают в пространстве пользователя (каждый в своём) и никаких проблем при добавлении драйверов к системе не возникает. Эта особенность называется динамическая реконфигурация, и она же обеспечивает **экстремально высокую** надежность и устойчивость микроядерных систем по сравнению с моноядерными: вышедший из строя драйвер можно перезагрузить, не останавливая систему.

В оставшейся части статьи будет рассматриваться только макроядерная архитектура, и, под операционной системой будет пониматься макроядерная операционная система. В макроядерной архитектуре, к которой принадлежит и ОС Linux, все услуги для прикладного приложения выполняют отдельные ветви кода внутри ядра. Приложение А, когда ему необходим доступ к определенному сервису, формирует **системный вызов** к ядру, который передается через единую для всех типов запросов точку входа. Код ядра выполняет обслуживание полученного системного вызова **в пространстве ядра**, и по завершению копирует результат в адресное пространство процесса А. В данном типе архитектуры значительно больше системных вызовов (порядка нескольких сотен), чем в микроядре. В Linux эту информацию можно найти в файле /usr/include/asm/unistd.h (unistd_32.h или unistd_64.h, в зависимости от архитектуры процессора), и это будет, скорее всего, около 300 вызовов.

Эта модель отлично работает до определенного этапа развития системы, как было в ранних версиях ядра Linux, где любое расширение функциональности ядра достигалось за счет создания новой **сборки** ядра. Для системы промышленного уровня, когда необходимо обеспечить поддержку широчайшего спектра оборудования, часть которого еще неизвестна в момент написания ядра, эта технология уже не подходит. Поэтому рано или поздно, но **любая** монолитная операционная система должна начать включать в себя ту или иную модель динамической реконфигурации. Для Linux это — технология модулей ядра, которая появилась, начиная с ядра 0.99 (1992г.) благодаря Питеру Мак-Дональду (Peter MacDonald).

Ядро и модуль

Механизм модулей представляет собой формальный список правил, согласно которому программный код размещается в адресном пространстве ядра и может использовать имена (символы) из пространства ядра: вызывать функции или использовать объекты данных. Таким образом, код модуля ядра становится неотъемлемой составной частью ядра и

выполняется с привилегиями суперпользователя или супервизора (для Intel x86 архитектуры — это выполнение в 0-м кольце защиты процессора) или со всеми возможными для ядра привилегиями. Это значит, что необнаруженная ошибка в коде любого динамически загруженного модуля может оказаться критической и привести не только к повреждению работающего ядра, но и повредить перманентные данные на дисковой и файловой системах, что сделает невозможным восстановление системы впоследствии. (Стоит отметить, что подобное развитие событий маловероятно, но маловероятно не значит абсолютно невозможно.)

В модулях Linux механизм связывания имён — это связывание по прямому **абсолютному** адресу символа в адресном пространстве ядра. В силу этого внесение минимальных изменений в ядро (компиляция ядра с изменённым конфигурационным параметром) приводит к смещению абсолютных адресов и **недопустимости** использования ранее подготовленных модулей. При внесении изменений в ядро все модули должны быть перекомпилированы. Чтобы обнаружить и запретить использование некорректных модулей в ядро вводятся жёсткие методы контроля, независящие от пользователя (это было продемонстрировано в начальных экспериментах, когда созданный модуль пытались загрузить на другом компьютере).

Заключение

Получив начальные сведения о схемах взаимодействия между модулем и ядром Linux, можно переходить к изучению системных вызовов, рассматривающихся в следующей статье.

Загрузка

Описание	Имя	Размер
	log_level.tgz	1KB

Об авторе

Олег Цилюрик



Олег Иванович Цилюрик, много лет был разработчиком программного обеспечения в крупных центрах разработки: ВНИИ РТ, НПО "Дельта", КБ ПМ. Последние годы работал над проектами в области промышленной автоматики, IP телефонии и коммуникаций. Автор нескольких книг. Преподаватель тренингового отделения международной софтверной компании Global Logic.

© Copyright IBM Corporation 2012
(www.ibm.com/legal/copytrade.shtml)
Торговые марки
(www.ibm.com/developerworks/ru/ibm/trademarks/)